

Build Task — Project 3: LangGraph App — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real production graph handles the messy edge cases). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The shape, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The shape, and the exact pieces

Basic Version

Project 3 is really the five practice exercises above, wired into one continuous graph instead of five separate scripts. Nothing here is a new idea — it's assembly.

Think about the shape first, before any code: a task comes in, the graph decides if it needs to search, it searches if so, it reasons over whatever it found (or admits it found nothing), then — before doing anything that can't be undone — it pauses and asks a human to approve, and only then finishes.

Try drawing this as boxes and arrows on paper first. Where does branching happen? Where does the pause happen?

Things to use:

- Your Doc09 graph's `StateGraph`, node/edge setup, and checkpointers.
- Your `08_rag.retrieve()` function, wrapped as a tool (from the `search_as_tool` exercise).
- `add_conditional_edges` for both the "need search or not" decision and the "found anything or not" decision.
- `interrupt()` and `Command(resume=...)` for the approval pause.
- A state shape (a `TypedDict` or similar) that has room for: the task, search results/sources, a draft, an approval decision, and the final answer.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Think of Project 3 as your Doc09 graph skeleton, with three additions layered on, each one already practiced separately above:

1. A search node/tool from `search_as_tool`, routed conditionally using `conditional_search`'s pattern.
2. A reasoning node that handles both "search found something" and "search found nothing" (`empty_search`'s pattern) — it should never silently guess when grounding is missing.
3. An approval node using `approval_pause`'s `interrupt()` / `Command(resume=...)` pattern, placed right before whatever the "risky action" is in your version (it can be a placeholder action — sending an email, updating a record, anything you choose to represent it).

The graph also needs a real checkpointer (not the in-memory placeholder you might have used for quick tests) so the pause/resume cycle is genuinely durable, per `search_failure`'s lesson about state surviving between steps.

The exact pieces:

- **State shape** — design this first, on paper. It needs: `task: str`, `sources: list`, `draft: str`, `approved: bool`, `answer: str`, at minimum. Getting this shape right before writing nodes saves you from constantly reshaping it mid-build.
- **Two conditional edges, not one** — the "route to search or skip" decision (right after the task comes in) and the "search found something or nothing" decision (right after search runs) are two separate decisions, at two separate points in the graph. Don't try to collapse them into one.
- **The approval node's placement** — it needs to run *after* the reasoning node produces a draft, but *before* whatever "the risky action" is in your version. It receives the draft and sources, not just the raw task, since that's what the human actually needs to judge.
- **The checkpointer** — use `MemorySaver` for local testing; note in your code (a comment is fine) that production would use a persistent one (like `SQLiteSaver` or a Postgres-backed one), since `MemorySaver` loses everything on restart.

Now try building the graph's node and edge structure completely, with placeholder node bodies (`pass` or a simple `return`), before writing any real logic inside them.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Each practice exercise above got an Advanced Version of its own — a hardened search tool, a hybrid router, a relevance threshold for empty search, a retry loop for search failures, and an audit trail for approvals. The Build Task's real Advanced-level question is: **do these five hardening ideas compose cleanly into one graph, or do they interfere with each other?**

They mostly compose cleanly, but two interactions are worth thinking through deliberately, not just bolting all five pieces on and hoping:

- **Retry (from `search_failure`) and the "found nothing" check (from `empty_search`) need to run in the right order.** A transient failure and "genuinely no relevant documents" are different problems with different correct responses — retrying a genuinely-empty result forever accomplishes nothing, and treating a timeout as "no grounding" throws away information (a retry might have succeeded). Your search node should exhaust its retries *first*, and only once it has a real result (even an empty one) should `has_results`-style routing decide what to do with it.
- **The approval payload (from `approval_pause`) should reflect what actually happened upstream**, not just the final draft. If search hit its relevance threshold and fell back to `cant_answer_node`'s honest message, the human reviewing at the approval step should see that this was an ungrounded response, not a normal one — otherwise the approval step becomes a rubber stamp instead of a real check.

The real design question underneath the whole Build Task isn't "did I add all five hardening features" — it's "does a human looking at the approval pause have an honest, complete picture of what the graph actually did to get here: did it search, did it find anything good, did it have to retry, is this grounded or not?"

One extra piece worth adding at this level: a `path` list in your state (e.g. `state["path"] = ["searched", "found_grounding", "awaiting_approval"]`), appended to by every node as it runs, and shown alongside the draft at the approval pause — this is what turns "trust the final answer" into "see exactly how the graph got here," which is this whole document's stated Goal.

Sketch what you'd add to your state shape and your approval payload to make the path fully visible, before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both wire the five pieces together as if each works in isolation, exactly as practiced. Advanced asks how they interact once combined — search's retry logic has to run to completion before the "found nothing" check means anything, and the approval step needs to honestly reflect whether the draft is genuinely grounded or a fallback — plus adds a visible `path` trail so a human approving something can see the graph's actual journey, not just its destination.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
state has: task, sources, draft, approved, answer

nodes:
    router_node      -- just passes the task through
    search_node      -- calls retrieve(), saves results into sources
    reason_node       -- builds a draft from sources (or a "no grounding" mess
    approval_node    -- interrupts, waits for a human decision
    finish_node       -- builds the final answer

edges:
    router_node -> (conditional: search or skip) -> search_node or reason_node
    search_node -> (conditional: found something or not) -> reason_node or reason
    reason_node -> approval_node
    approval_node -> finish_node

run it:
    a search-needing task -> should pause at approval_node -> resume -> finish
    a task with no matching documents -> reason_node makes an honest "can't answer"
```

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
class ProjectState(TypedDict):
    task: str
    sources: list
    draft: str
    approved: bool
    answer: str

def route_search(state) -> str:
    # keyword check or small model call, from conditional_search exercise
    ...

def search_node(state) -> dict:
    # calls retrieve(), from search_as_tool exercise
    ...

def has_grounding(state) -> str:
    # from empty_search exercise
    ...

def reason_node(state) -> dict:
    # builds draft from sources, or an honest "can't ground this" draft if source
    ...

def approval_node(state) -> dict:
    # interrupt() with draft + sources, from approval_pause exercise
    ...

def finish_node(state) -> dict:
    if state["approved"]:
        return {"answer": state["draft"]}
    return {"answer": "Not approved -- no action taken."}

builder = StateGraph(ProjectState)
builder.add_node("router", lambda state: state)
builder.add_node("search_node", search_node)
builder.add_node("reason_node", reason_node)
builder.add_node("approval_node", approval_node)
builder.add_node("finish_node", finish_node)

builder.add_conditional_edges("router", route_search, {"search": "search_node", "
```

```
builder.add_conditional_edges("search_node", has_grounding, {"no_grounding": "reason_node", "grounding": "approval_node"})
builder.add_edge("reason_node", "approval_node")
builder.add_edge("approval_node", "finish_node")

graph = builder.compile(checkpointer=MemorySaver())
```

Note both conditional-edge branches from `search_node` land on `reason_node` here — the difference is what `reason_node` finds in `state["sources"]` when it runs (empty vs. populated), which is what decides whether the draft is a real answer or an honest "can't answer."

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
class ProjectState(TypedDict):
    task: str
    sources: list
    grounded: bool
    draft: str
    approved: bool
    answer: str
    path: list[str]  # a visible trail of what the graph actually did

def search_node(state):
    for up to MAX_ATTEMPTS:
        try: call retrieve(), return sources + append "searched" to path
        except a transient error: wait and retry, unless out of attempts
        except a permanent error: append "search_failed" to path, re-raise

def has_grounding(state):
    if sources is empty or best score is below threshold:
        append "no_grounding" to path
        return "no_grounding"
    append "found_grounding" to path
    return "reason"

def approval_node(state):
    payload = draft, sources, grounded flag, and the full path so far
    decision = interrupt(payload)
    append "approved" or "rejected" to path
    return approved flag + who/when, from the decision
```

Here's almost the whole thing — fill in the missing piece yourself:

```
from datetime import datetime, timezone
from langgraph.types import interrupt

MAX_ATTEMPTS = 3
TRANSIENT_ERRORS = (TimeoutError, ConnectionError)
RELEVANCE_THRESHOLD = 0.75

def search_node(state: ProjectState) -> dict:
```

```

path = state.get("path", []) + []
for attempt in range(MAX_ATTEMPTS):
    try:
        scored_sources = retrieve_with_scores(state["task"])
        path.append("searched")
        return {"sources": scored_sources, "path": path}
    except TRANSIENT_ERRORS:
        if attempt == MAX_ATTEMPTS - 1:
            path.append("search_failed")
            raise
return {"sources": [], "path": path}

def has_grounding(state: ProjectState) -> str:
    scored_sources = state.get("sources", [])
    best_score = max((score for _, score in scored_sources), default=0.0)
    if best_score < RELEVANCE_THRESHOLD:
        return "no_grounding"
    return "reason"

def approval_node(state: ProjectState) -> dict:
    # your turn: build the interrupt() payload from state["draft"],
    # state["sources"], state["grounded"], and state["path"] -- this is
    # what makes the approval step honest about what actually happened,
    # not just a draft with no context -- then handle the returned
    # decision dict the same way approval_pause's Advanced Version does
    ...

```

Fill in `approval_node` yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate's pseudocode and near-complete code wire the five pieces together assuming each one always works cleanly on the first try. Advanced adds the retry loop from `search_failure`, the relevance threshold from `empty_search`, and a `path` field every node appends to — so the approval step (and anyone debugging a run later) can see the graph's actual journey, including any retries or fallbacks, not just its final draft.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Read all three depths — they're not "wrong, less wrong, right," they're 3 real, valid ways to solve the same problem, with real tradeoffs between them. Every example assumes a `.env`-configured model and a `retriever.py` reused from `08_rag`.

Basic Version

Approach 1 — the direct way, all five pieces wired together

```
# state.py
from typing import TypedDict

class ProjectState(TypedDict):
    task: str
    sources: list
    draft: str
    approved: bool
    answer: str
```

```
# nodes.py
from retriever import retrieve

def route_search(state):
    task = state["task"].lower()
    if "document" in task or "policy" in task:
        return "search"
    return "skip"

def search_node(state):
    results = retrieve(state["task"])
    return {"sources": results}

def reason_node(state):
    if not state["sources"]:
        return {"draft": "I don't have grounding for this in my documents."}
    context = "\n\n".join(s.page_content for s in state["sources"])
    draft = model.invoke("Context: " + context + "\n\nTask: " + state["task"])
    return {"draft": draft.content}
```

```
def approval_node(state):
    decision = interrupt({"draft": state["draft"]})
    return {"approved": decision == "approved"}

def finish_node(state):
    if state["approved"]:
        return {"answer": state["draft"]}
    return {"answer": "Not approved -- no action taken."}
```

```
# graph.py
from langgraph.graph import StateGraph
from langgraph.checkpoint.memory import MemorySaver
from state import ProjectState
from nodes import route_search, search_node, reason_node, approval_node, finish_n

builder = StateGraph(ProjectState)
builder.add_node("search_node", search_node)
builder.add_node("reason_node", reason_node)
builder.add_node("approval_node", approval_node)
builder.add_node("finish_node", finish_node)

builder.set_conditional_entry_point(route_search, {"search": "search_node", "skip
builder.add_edge("search_node", "reason_node")
builder.add_edge("reason_node", "approval_node")
builder.add_edge("approval_node", "finish_node")

graph = builder.compile(checkpointer=MemorySaver())
```

This meets every Build Task requirement: routed search, an honest "no grounding" draft, a real approval pause, and durable state. It's missing type hints, and it doesn't yet handle a `retrieve()` failure or a stale approval — both fine to leave out of a first working version.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints, `main.py` 's visible pause/resume loop

```
# state.py
from typing import TypedDict

class ProjectState(TypedDict):
    task: str
    sources: list
    draft: str
    approved: bool
    answer: str
```

```
# nodes.py
from langgraph.types import interrupt
from retriever import retrieve

def route_search(state: ProjectState) -> str:
    task = state["task"].lower()
    keywords = ["document", "policy", "according to"]
    if any(keyword in task for keyword in keywords):
        return "search"
    return "skip"

def search_node(state: ProjectState) -> dict:
    results = retrieve(state["task"])
    return {"sources": results}

def reason_node(state: ProjectState) -> dict:
    if not state["sources"]:
        return {"draft": "I don't have grounding for this in my documents."}

    context = "\n\n".join(source.page_content for source in state["sources"])
    prompt = f"Using this context, answer the task.\n\nContext:\n{context}\n\nTask: {state['task']}\n\nAnswer:"
    response = model.invoke(prompt)
    return {"draft": response.content}
```

```
def approval_node(state: ProjectState) -> dict:
    decision = interrupt({"draft": state["draft"], "sources": state["sources"]})
    return {"approved": decision == "approved"}

def finish_node(state: ProjectState) -> dict:
    if state["approved"]:
        return {"answer": state["draft"]}
    return {"answer": "Not approved -- no action taken."}
```

```
# graph.py
from langgraph.graph import StateGraph
from langgraph.checkpoint.memory import MemorySaver
from state import ProjectState
from nodes import route_search, search_node, reason_node, approval_node, finish_n

def build_graph():
    builder = StateGraph(ProjectState)
    builder.add_node("search_node", search_node)
    builder.add_node("reason_node", reason_node)
    builder.add_node("approval_node", approval_node)
    builder.add_node("finish_node", finish_node)

    builder.set_conditional_entry_point(
        route_search, {"search": "search_node", "skip": "reason_node"}
    )
    builder.add_edge("search_node", "reason_node")
    builder.add_edge("reason_node", "approval_node")
    builder.add_edge("approval_node", "finish_node")

    return builder.compile(checkpointer=MemorySaver())

graph = build_graph()
```

```
# main.py
from langgraph.types import Command
from graph import graph

def main() -> None:
    task = input("Enter a task: ")
```

```

config = {"configurable": {"thread_id": "run-1"}}

graph.invoke({"task": task}, config)
paused = graph.get_state(config)
print(f"Draft: {paused.values['draft']}")
print(f"Sources used: {len(paused.values.get('sources', []))}")

decision = input("Approve this answer? (yes/no): ")
resume_value = "approved" if decision.strip().lower() == "yes" else "rejected"
final = graph.invoke(Command(resume=resume_value), config)

print(f"Final answer: {final['answer']}")

if __name__ == "__main__":
    main()

```

Difference from Basic: full type hints throughout. `graph.py` wraps construction in `build_graph()` instead of module-level statements, matching how a real project separates "define the graph" from "run the graph." `main.py` now uses `graph.get_state(config)` to inspect the paused draft and source count, instead of relying on `invoke()`'s return value alone — and reports the final answer explicitly. This version still doesn't retry a failed search, cap what gets shown at approval, or distinguish a genuinely-empty search from a low-relevance one — that's what Advanced adds.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — retry on search, a relevance threshold, and a visible `path` at approval

```
# state.py
from typing import TypedDict

class ProjectState(TypedDict):
    task: str
    sources: list
    grounded: bool
    draft: str
    approved: bool
    approved_by: str | None
    answer: str
    path: list[str]
```

```
# nodes.py
import time
from langgraph.types import interrupt
from retriever import retrieve_with_scores

MAX_ATTEMPTS = 3
TRANSIENT_ERRORS = (TimeoutError, ConnectionError)
RELEVANCE_THRESHOLD = 0.75

def route_search(state: ProjectState) -> str:
    task = state["task"].lower()
    keywords = ["document", "policy", "according to"]
    if any(keyword in task for keyword in keywords):
        return "search"
    return "skip"

def search_node(state: ProjectState) -> dict:
    path = state.get("path", []) + []
    for attempt in range(MAX_ATTEMPTS):
        try:
            scored_sources = retrieve_with_scores(state["task"])
```

```

        path.append("searched")
        return {"sources": scored_sources, "path": path}
    except TRANSIENT_ERRORS:
        if attempt == MAX_ATTEMPTS - 1:
            path.append("search_failed")
            raise
        time.sleep(2 ** attempt)
    return {"sources": [], "path": path}

def has_grounding(state: ProjectState) -> str:
    scored_sources = state.get("sources", [])
    best_score = max((score for _, score in scored_sources), default=0.0)
    if best_score < RELEVANCE_THRESHOLD:
        return "no_grounding"
    return "reason"

def reason_node(state: ProjectState) -> dict:
    path = state.get("path", []) + []
    scored_sources = state.get("sources", [])
    best_score = max((score for _, score in scored_sources), default=0.0)

    if not scored_sources or best_score < RELEVANCE_THRESHOLD:
        path.append("no_grounding")
        return {
            "draft": "I don't have grounding for this in my documents.",
            "grounded": False,
            "path": path,
        }

    context = "\n\n".join(chunk.page_content for chunk, _ in scored_sources)
    prompt = f"Using this context, answer the task.\n\nContext:\n{context}\n\nTask: {task}"
    response = model.invoke(prompt)
    path.append("found_grounding")
    return {"draft": response.content, "grounded": True, "path": path}

def approval_node(state: ProjectState) -> dict:
    path = state.get("path", []) + []
    decision = interrupt({
        "draft": state["draft"],
        "grounded": state["grounded"],
        "sources": [chunk.page_content[:200] for chunk, _ in state.get("sources", [])],
        "path_so_far": path,
        "prompt": "Approve this answer before it's sent? (approved/rejected)",
    })

```

```

approved = decision["decision"] == "approved"
path.append("approved" if approved else "rejected")
return {
    "approved": approved,
    "approved_by": decision.get("by"),
    "path": path,
}

```

```

def finish_node(state: ProjectState) -> dict:
    if state["approved"]:
        return {"answer": state["draft"]}
    return {"answer": "Not approved -- no action taken."}

```

```

# graph.py
from langgraph.graph import StateGraph
from langgraph.checkpoint.memory import MemorySaver
from state import ProjectState
from nodes import (
    route_search,
    search_node,
    has_grounding,
    reason_node,
    approval_node,
    finish_node,
)

def build_graph():
    builder = StateGraph(ProjectState)
    builder.add_node("search_node", search_node)
    builder.add_node("reason_node", reason_node)
    builder.add_node("approval_node", approval_node)
    builder.add_node("finish_node", finish_node)

    builder.set_conditional_entry_point(
        route_search, {"search": "search_node", "skip": "reason_node"}
    )
    builder.add_conditional_edges(
        "search_node", has_grounding, {"no_grounding": "reason_node", "reason": "reason_node"}
    )
    builder.add_edge("reason_node", "approval_node")
    builder.add_edge("approval_node", "finish_node")

    # NOTE: MemorySaver loses everything on restart -- a real deployment
    # would use SqliteSaver or a Postgres-backed checkpointer instead.

```

```

        return builder.compile(checkpointer=MemorySaver())

graph = build_graph()

```

```

# main.py
from datetime import datetime, timezone
from langgraph.types import Command
from graph import graph

def main() -> None:
    task = input("Enter a task: ")
    config = {"configurable": {"thread_id": "run-1"}}

    graph.invoke({"task": task}, config)
    paused = graph.get_state(config)
    print(f"Path so far: {paused.values['path']}")
    print(f"Grounded: {paused.values['grounded']}")
    print(f"Draft: {paused.values['draft']}")

    reviewer = input("Your email: ")
    decision = input("Approve this answer? (yes/no): ")
    resume_value = {
        "decision": "approved" if decision.strip().lower() == "yes" else "rejecte
        "by": reviewer,
        "at": datetime.now(timezone.utc).isoformat(),
    }
    final = graph.invoke(Command(resume=resume_value), config)

    print(f"Final path: {final['path']}")
    print(f"Final answer: {final['answer']}")

if __name__ == "__main__":
    main()

```

Approach 2 — same structure, `pydantic` for a validated state instead of `TypedDict`

`TypedDict` (Approach 1) gives you type hints but no runtime checking — a node can still return `{"approved": "yes"}` (a string, not a bool) and nothing catches it until something downstream breaks confusingly. A `pydantic` model validates every field on construction.

```

# state.py
from pydantic import BaseModel

```

```
class ProjectState(BaseModel):
    task: str
    sources: list = []
    grounded: bool = False
    draft: str = ""
    approved: bool = False
    approved_by: str | None = None
    answer: str = ""
    path: list[str] = []
```

Everything else stays the same shape as Approach 1 — nodes still read and return dicts that LangGraph merges into state, but now any field with the wrong type (an `approved` that isn't genuinely a `bool`, for example) raises a clear `pydantic.ValidationError` immediately, at the point state is constructed, instead of surfacing later as a confusing `if state["approved"]:` bug three nodes downstream.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's graph works correctly on the happy path but trusts `retrieve()` never to fail, trusts any non-empty result list to mean real grounding, and shows the approval step only a bare draft with no record of how it got there. Approach 1 fixes all three: a retry loop around search, a relevance threshold instead of an empty-list check, and a `path` trail plus an audited resume payload at the approval step, so a human reviewer sees the graph's actual journey. Approach 2 doesn't change any of that behavior — it swaps the state container from `TypedDict` (hints only) to `pydantic.BaseModel` (hints enforced at runtime), catching a wrong-typed field the moment it's written instead of whenever it happens to cause a visible bug.

Which one should you actually write? For the Build Task as scoped, Intermediate already satisfies every stated requirement — ship that first, and confirm the full test-case table in the exercise README passes before adding anything else. Reach for Advanced Approach 1's retry-plus-threshold-plus-path once you're running this against a real, occasionally-flaky search backend and real human reviewers who need to trust what they're approving — which, per this document's own Goal, is the whole point of Project 3. Reach for Approach 2's `pydantic` state only once a wrong-typed state field has actually caused a confusing bug, or once this graph is being extended by more than one person and a runtime check on every field is worth the small added dependency.